

K-china

FIEI

EQUIPO

Aviles Cortez, Juan Diego

Benites Berrocal, Carlos

Koo Benavides, Armando Cesar

Suazo Apolinario, Andrea del Pilar

Pacheco Diaz, Michael Danner

Problemática

En la UNFV, los componentes electrónicos terminan en desuso al final de cada ciclo. Su intercambio informal en redes sociales genera graves problemas.



Riesgo de fraudes

Sin garantía de identidad del vendedor, los intercambios en grupos de Facebook exponen a estafas entre estudiantes.



Falta de información técnica

Imposible clasificar módulos bajo parámetros críticos: voltajes, estado de soldadura y protocolos de comunicación.



Freno a la economía circular

Se debilita el ecosistema de reutilización dentro de la FIEI, generando desperdicio de componentes funcionales.

Objetivos del Proyecto

 **Objetivo General:** Desarrollar e implementar un sistema C2C transaccional escalable (HardSwap FIEI) para facilitar la comercialización estructurada y segura de hardware de desarrollo entre los estudiantes de la UNFV.

01

Diseñar una BD relacional robusta capaz de gestionar especificaciones técnicas y validar credenciales académicas.

02

Implementar un backend y frontend escalable con Flutter + Supabase que garantice latencia mínima y consultas eficientes.

03

Implementar mecanismos de garantía estructural (validación @unfv.edu.pe, perfiles verificados) para evitar el fraude.

Stack Tecnológico



Flutter / Dart

Frontend · App Móvil

- Compilación nativa (Android)
- UX interactiva de alta calidad Gestión de estados complejos 60 fps garantizados



Supabase

Backend · PostgreSQL

- APIs autogeneradas en la nube
- Autenticación segura integrada
- Escalabilidad inmediata
- Trigger automático de usuarios



Row Level Security

Seguridad · RLS

- Cada usuario solo edita sus datos
- `auth.uid() = user_id` validado en BD
- Políticas a nivel de motor SQL
- Imposible manipular datos ajenos

Conexión Frontend-Backend

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:supabase_flutter/supabase_flutter.dart';
import 'core/router/app_router.dart';
import 'core/theme/app_theme.dart';
/// Punto de entrada — K-china FIEI
///
/// Orden de inicialización:
/// 1. Flutter binding
/// 2. Conexión a Supabase
/// 3. ProviderScope (Riverpod)
/// 4. MaterialApp.router con go_router
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  // Inicializar conexión a Supabase
  await Supabase.initialize(
    url: 'https://tcntyolvhafxkqilrfoc.supabase.co',
    anonKey:
'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InRjb250b2x2aGFmeGtxaWxyZm9jliwicm9sZSI6ImFub24iLCJpYXQiOiE3NzgxNjY2MzIsImV4cCI6MjA5Mzc0MjYzMn0.CbZENjg7ql-zXIXaDXz_MlwuoOnmiua4w1QNACBTwRk',
  );
  runApp(
    // ProviderScope es el contenedor de toda la inyección de dependencias
    const ProviderScope(
      child: KChinaFieiApp(),
    ),
  );
}
```

```
class KChinaFieiApp extends ConsumerWidget {
  const KChinaFieiApp({super.key});
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final router = ref.watch(appRouterProvider);
    return MaterialApp.router(
      title: 'K-china FIEI — UNFV',
      debugShowCheckedModeBanner: false,
      // Temas Material 3
      theme: AppTheme.light,
      darkTheme: AppTheme.dark,
      themeMode: ThemeMode.system,
      // Router con guards de autenticación
      routerConfig: router,
    );
  }
}
```


Conexión Frontend-Backend

// Convierte imágenes locales a Base64 (comprimidas por image_picker)

```
final base64Images = <String>[];
for (final path in imageUrls.take(3)) {
  final file = File(path);
  if (await file.exists()) {
    final bytes = await file.readAsBytes();
    base64Images.add(base64Encode(bytes));
  }
}
```

// Obtener el UUID real de la categoría mediante consulta asíncrona

```
final catData = await _supabase.from('categories').select('id').eq('name', categoryName).single();
final realCategoryId = catData['id'] as String;
```

// Inserción asíncrona directa a PostgreSQL vía Supabase SDK

```
final data = await _supabase.from('products').insert({
  'title':  title,
  'description': description,
  'price':  priceInCentavos,
  'category_id': realCategoryId,
  'user_id':  userId,
  'images_base64': base64Images,
  'type':  type,
}).select('id').single();
...
```

FLUJO PASO A PASO



- Lee archivo físico.

la aplicación lee el archivo físico. Cuando el usuario toma la foto del componente electrónico, Flutter interactúa directamente con el sistema de archivos del celular (Android o iOS). Capturamos la foto y la cargamos directamente en la memoria RAM del dispositivo.



- Pasa a Base64.

Aquí está la optimización clave de nuestro MVP: en lugar de subir esa foto a un servidor de archivos tradicional (como Amazon S3), agarramos los bytes de la imagen y los codificamos en una cadena de texto infinita. Literalmente, convertimos la fotografía en letras y números desde el mismo celular.



- Inyecta JSON
Empaquetamos toda la información — el título, el precio, la categoría y nuestro texto Base64 gigante— en un solo objeto JSON. Usamos Supabase para inyectar todo este bloque directamente a nuestra base de datos PostgreSQL en la nube. ¿El resultado? Guardamos los datos y la imagen en una sola petición asíncrona, ahorrando peticiones HTTP y costos de almacenamiento

Seguridad Inquebrantable

```
-- =====
-- 1. SINCRONIZACIÓN AUTOMÁTICA DE USUARIOS (TRIGGER)
-- =====
```

-- Función que se ejecuta cada vez que alguien se registra en la app.

```
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS trigger AS $$
BEGIN
    -- Copia los datos del nuevo usuario desde el sistema de Auth de Supabase
    -- hacia nuestra tabla pública 'users' para poder enlazarle productos.
    INSERT INTO public.users (id, email, name, role)
    VALUES (
        new.id,
        new.email,
        -- Extrae el nombre de los metadatos, o usa la primera parte del correo
        COALESCE(new.raw_user_meta_data->>'name', split_part(new.email, '@', 1)),
        'user'
    );
    RETURN new;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

-- El 'Trigger' o gatillo: Vigila la tabla auth.users y dispara la función

```
CREATE TRIGGER on_auth_user_created
AFTER INSERT ON auth.users
FOR EACH ROW EXECUTE PROCEDURE public.handle_new_user();
```


Seguridad Inquebrantable

-- =====

2. TABLA DE PRODUCTOS (Lógica de negocio y optimización de costos)

-- =====

CREATE TABLE IF NOT EXISTS public.products (id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

-- Llave foránea: Si el usuario se borra, sus productos también (CASCADE)

user_id UUID NOT NULL REFERENCES public.users(id) ON DELETE CASCADE,

-- Restricciones de integridad: El precio no puede ser negativo o cero

price INTEGER NOT NULL CHECK (price > 0),

-- Estados controlados directamente por la BD (Evita errores del Frontend)

status TEXT NOT NULL DEFAULT 'AVAILABLE'

CHECK (status IN ('AVAILABLE', 'SOLD', 'EXPIRED')),

-- EL "HACK" DE ARQUITECTURA:

-- En vez de pagar Storage de AWS/Supabase, guardamos las fotos

-- convertidas a texto (Base64) en un Array []. ¡Ahorro total para el MVP!

images_base64 TEXT[] DEFAULT '{}',

-- Soporte para escalar: Permite vender un componente solo ('PRODUCT')

-- o un conjunto de cosas ('KIT')

type TEXT NOT NULL DEFAULT 'PRODUCT'

CHECK (type IN ('PRODUCT', 'KIT')));

Seguridad Inquebrantable

```
-- =====  
-- 3. ROW LEVEL SECURITY (RLS) - La base de datos se defiende sola  
-- =====  
-- Activamos el escudo en la tabla productos. A partir de ahora,  
-- nadie puede hacer NADA sin pasar por una política.  
ALTER TABLE public.products ENABLE ROW LEVEL SECURITY;  
-- REGLA 1 (Lectura): El catálogo es público. Cualquiera puede hacer un SELECT.  
CREATE POLICY "catalog is public" ON public.products FOR SELECT USING (true);  
-- REGLA 2 (Edición segura): "Solo el dueño puede editar su producto"  
-- auth.uid() extrae el ID del usuario directamente del token JWT seguro.  
-- Si un hacker intenta modificar un producto ajeno, PostgreSQL lo bloquea  
-- porque su auth.uid() no coincidirá con el user_id del producto.  
CREATE POLICY "owners can update products" ON public.products FOR UPDATE USING (auth.uid() = user_id);  
-- REGLA 3 (Borrado seguro): Misma lógica para eliminar.  
CREATE POLICY "owners can delete products" ON public.products FOR DELETE USING (auth.uid() = user_id);
```

Sprint 1 - Historias de Usuario

Historia de usuario PB-01 - Registro con correo institucional

Como estudiante FIEI

quiero registrarme con mi correo institucional

para validar que pertenezco a la universidad y generar confianza dentro de la plataforma.

Validación:

- Verificar dominio @unfv.edu.pe en el formulario.
- Enviar link de activación al correo ingresado.
- Denegar registro con correos genéricos (Gmail, Outlook).

Sprint:

Sprint 1

Prioridad:

Alta

Estimación:

12h

Historia de usuario PB-02 - Publicación con imágenes reales

Como vendedor

quiero publicar componentes electrónicos con imágenes reales

para mostrar el estado físico del producto.

Validación:

- Permitir carga de hasta 3 fotos por publicación.
- Validar formatos permitidos (JPG, PNG) y peso máximo.
- Previsualizar imágenes antes de publicar el anuncio.

Sprint:

Sprint 1

Prioridad:

Alta

Estimación:

16h

Historia de usuario PB-03 - Información técnica y datasheet

Como vendedor

quiero agregar información técnica del componente (modelo, estado y datasheet)

para que los compradores puedan verificar compatibilidad.

Validación:

- Campos obligatorios para modelo y estado (1 al 10).
- Campo opcional para insertar link a PDF de Datasheet.
- Visualizar esta información en la ficha del producto.

Sprint:

Sprint 1

Prioridad:

Alta

Estimación:

10h

Historia de usuario PB-04 - Filtros y búsqueda de componentes

Como comprador

quiero buscar componentes mediante filtros y categorías

para encontrar rápidamente lo que necesito.

Validación:

- Barra de búsqueda predictiva por nombre de componente.
- Filtros por categoría (Microcontroladores, Sensores, etc.).
- Ordenar resultados por precio de menor a mayor.

Sprint:

Sprint 1

Prioridad:

Alta

Estimación:

18h

Historia de usuario PB-05 - Detalle y fotografía del producto

Como comprador

quiero visualizar detalles técnicos y fotografías del producto

para evaluar el componente antes de contactar al vendedor.

Validación:

- Galería de imágenes expandible al hacer clic.
- Mostrar descripción técnica completa y clara.
- Botón destacado para Iniciar Chat / Contactar.

Sprint:

Sprint 1

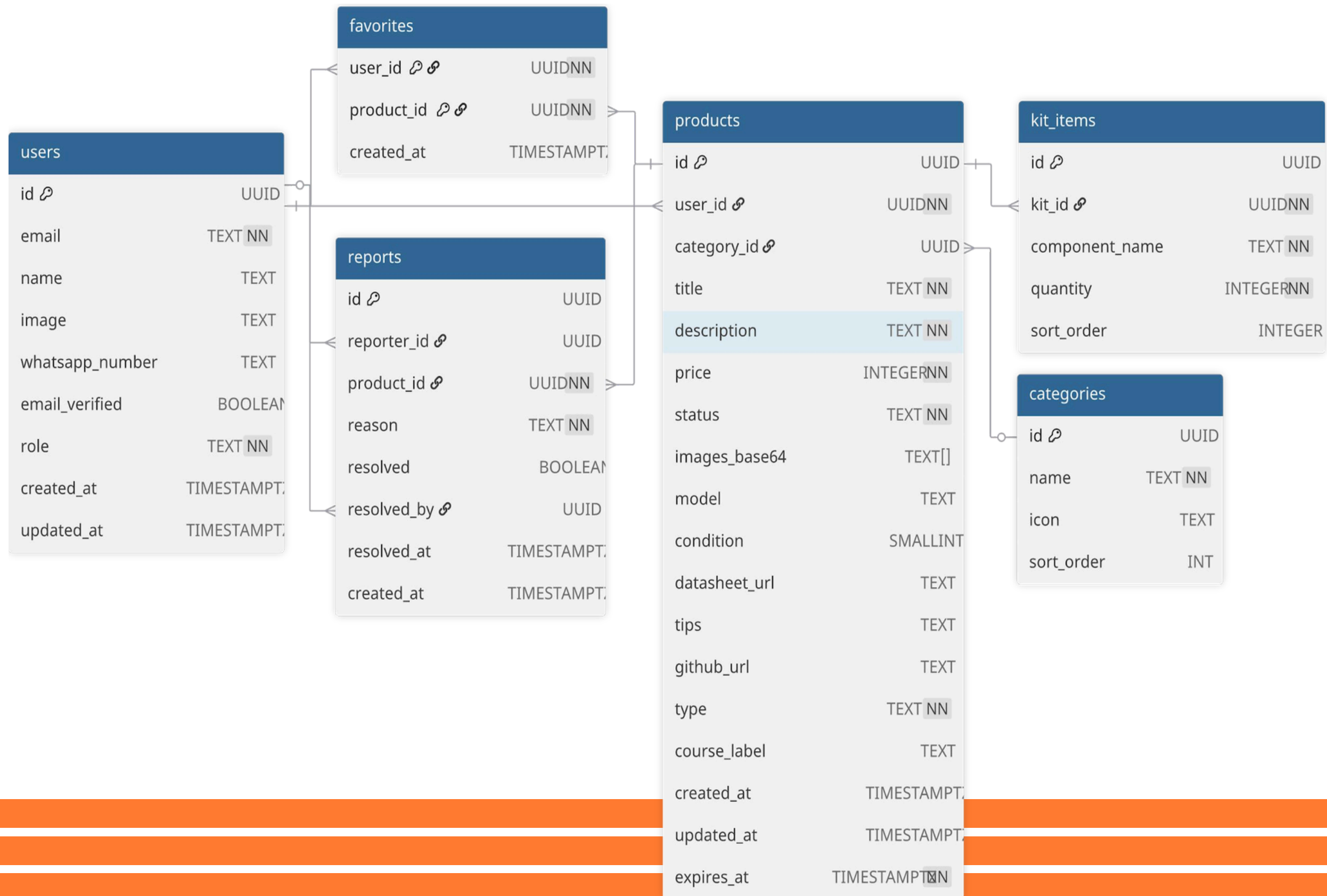
Prioridad:

Alta

Estimación:

8h

Diseño de la Base de Datos



Espejo Auth

Trigger automático sincroniza auth.users
→ public.users

Imágenes en Array

images_base64 TEXT[] evita tablas
intermedias

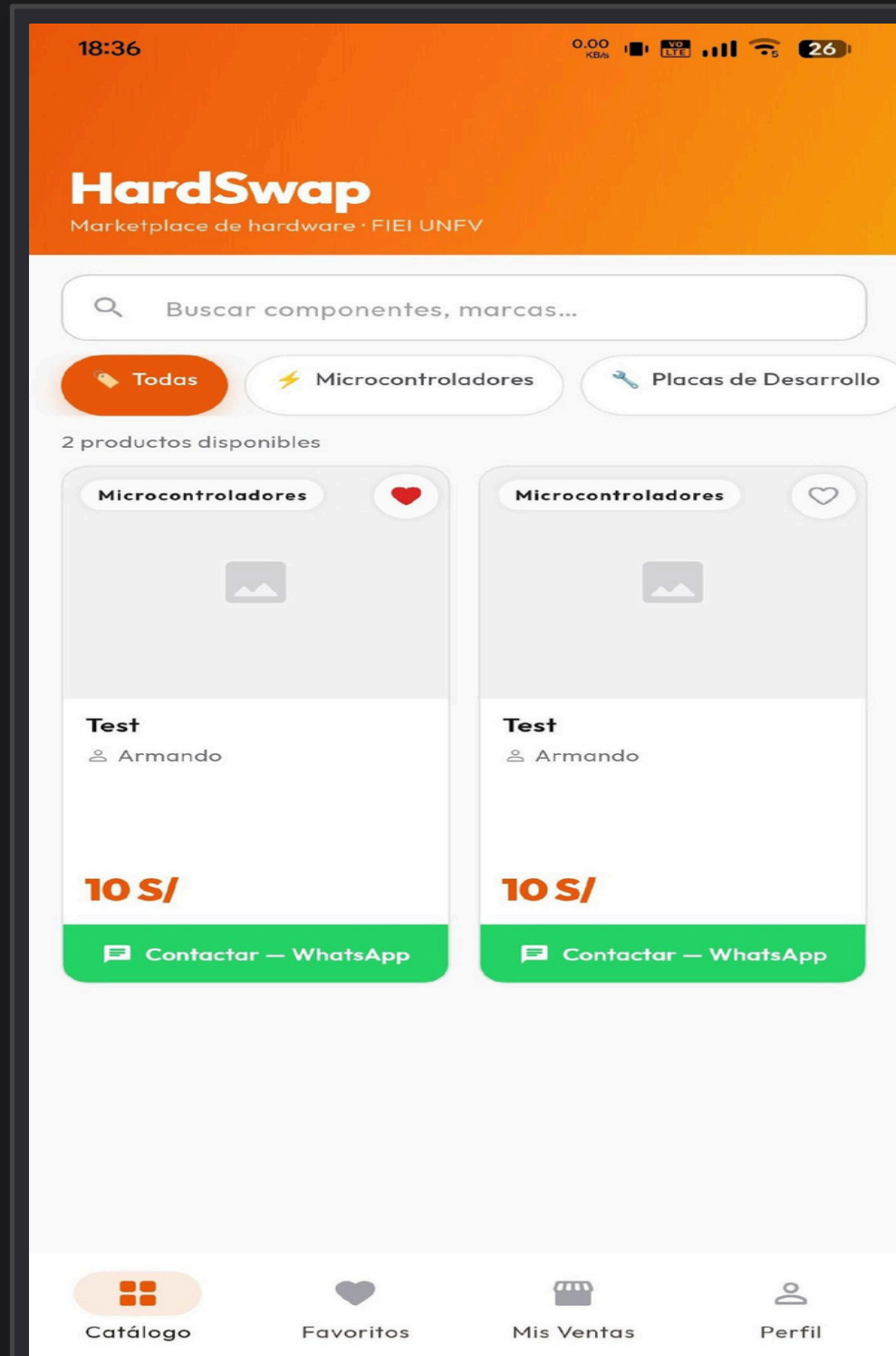
Kits escalables

kit_items con ON DELETE CASCADE para
integridad

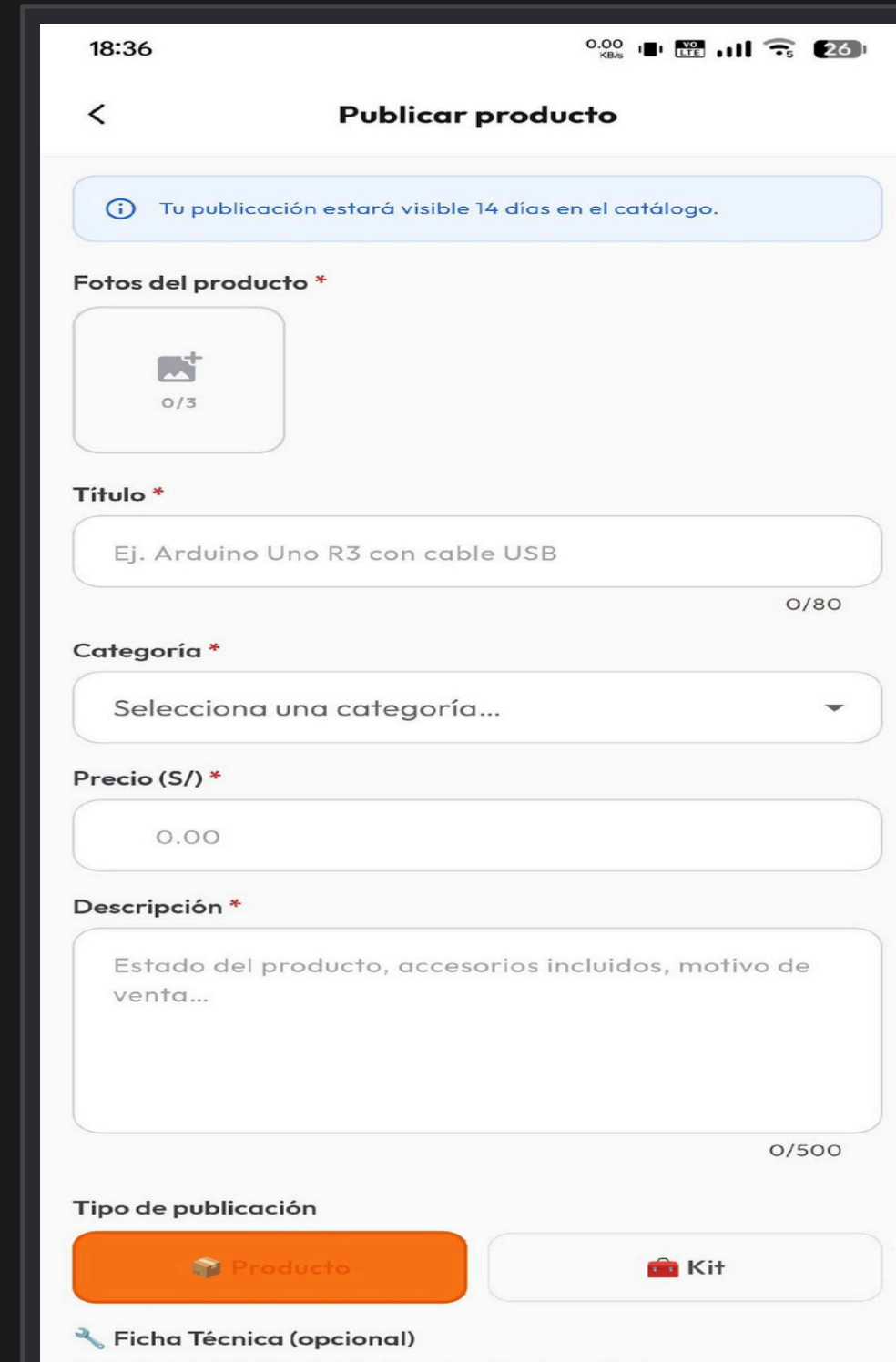
Seguridad RLS

auth.uid() = user_id en UPDATE/DELETE
de productos

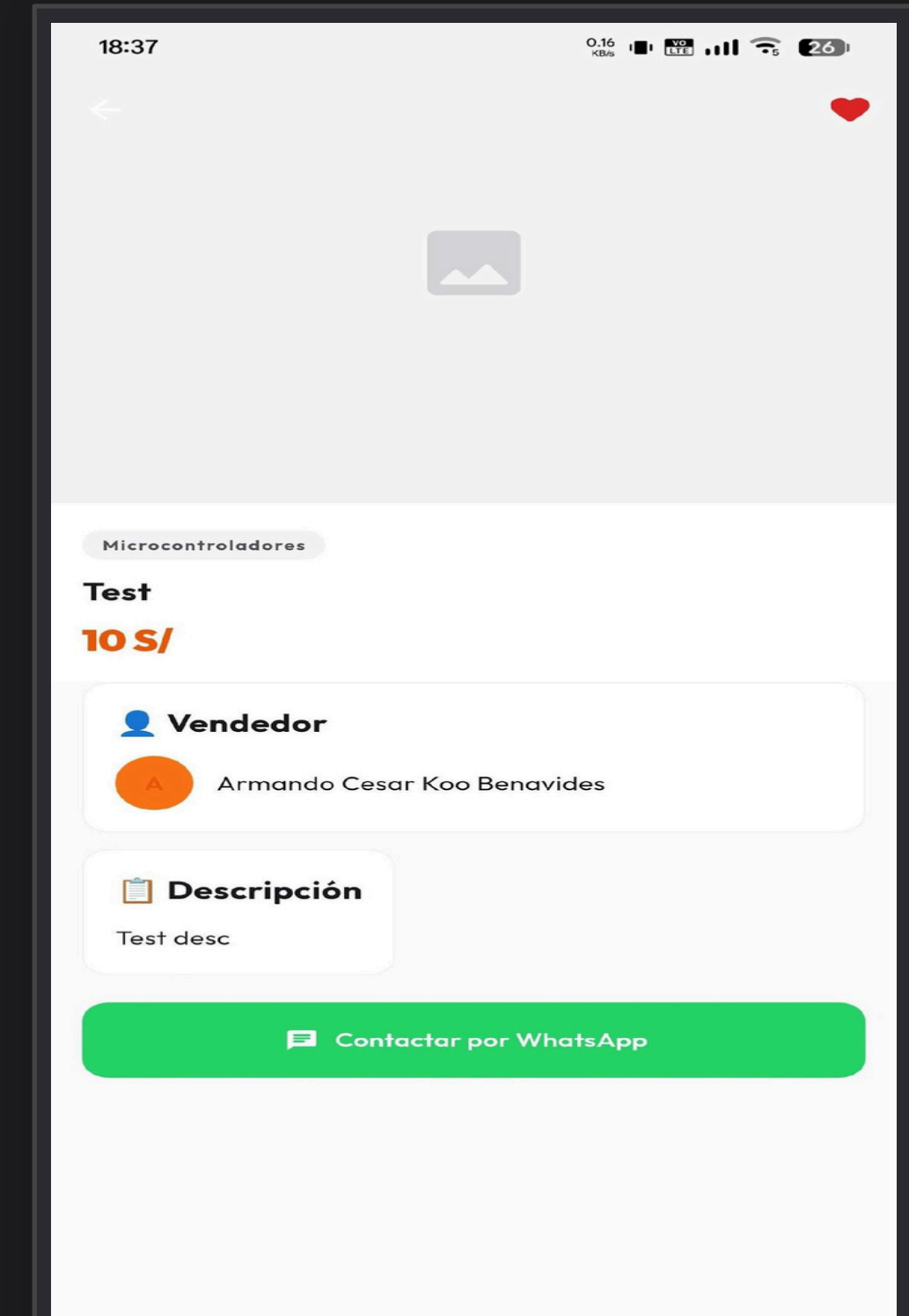
Sprint 1 · App en Acción



Catálogo principal



Publicar producto



Detalle del producto

Sprint 2 · Historias de Usuario

Parte 4 · Aviles Cortez, Juan Diego

Historia de usuario

PB-06 - Guardar en favoritos

Como

usuario

quiero

guardar publicaciones favoritas

para

revisarlas posteriormente.

Validación:

▪ Icono de corazón habilitado en cada tarjeta de producto.

▪ Crear vista de "Mis Favoritos." en el panel de perfil.

▪ Remover publicación de la lista al desmarcar el icono.

Sprint:

Sprint 2

Prioridad:

Baja

Estimación:

6h

Historia de usuario

PB-07 - Puntos de entrega seguros

Como

estudiante

quiero

acordar puntos de entrega seguros dentro de la universidad

para

facilitar el intercambio presencial.

Validación:

▪ Opciones predefinidas de encuentro (Ej. Patio FIEI, Cafetería).

▪ Integrar punto acordado dentro del chat interno.

▪ Notificación de confirmación de punto de entrega.

Sprint:

Sprint 2

Prioridad:

Media

Estimación:

14h

Historia de usuario

PB-08 - Gestión de mis publicaciones

Como

usuario

quiero

editar o eliminar mis publicaciones

para

actualizar el catálogo cuando el producto ya no esté disponible.

Validación:

▪ Botón de editar precio y descripción en publicaciones activas.

▪ Botón de "Marcar como vendido" para ocultar del catálogo.

▪ Ventana de confirmación antes de eliminar el producto.

Sprint:

Sprint 2

Prioridad:

Media

Estimación:

10h

Flujo del Sistema — Sprint 2

1

Comprador guarda favoritos con el ícono  (PB-06)

2

Se contacta al vendedor y se coordina punto seguro UNFV (PB-07)

3

Vendedor cierra el ciclo: elimina o marca como vendido (PB-08)

Quiero construir una plataforma de compra y venta de componentes electrónicos para estudiantes de la UNFV

Registro con correo institucional

Como estudiante FIEI quiero registrarme con mi correo institucional para validar que pertenezco a la universidad y generar confianza dentro de la plataforma.

Publicación con imágenes reales

Como vendedor quiero publicar componentes electrónicos con imágenes reales para mostrar el estado físico del producto.

Información técnica y datasheet

Como vendedor quiero agregar información técnica del componente (modelo, estado y datasheet) para que los compradores puedan verificar compatibilidad.

Filtros y búsqueda de componentes

Como comprador quiero buscar componentes mediante filtros y categorías para encontrar rápidamente lo que necesito.

Detalle y fotografía del producto

Como comprador quiero visualizar detalles técnicos y fotografías del producto para evaluar el componente antes de contactar al vendedor.

Guardar en favoritos

Como usuario quiero guardar publicaciones favoritas para revisarlas posteriormente.

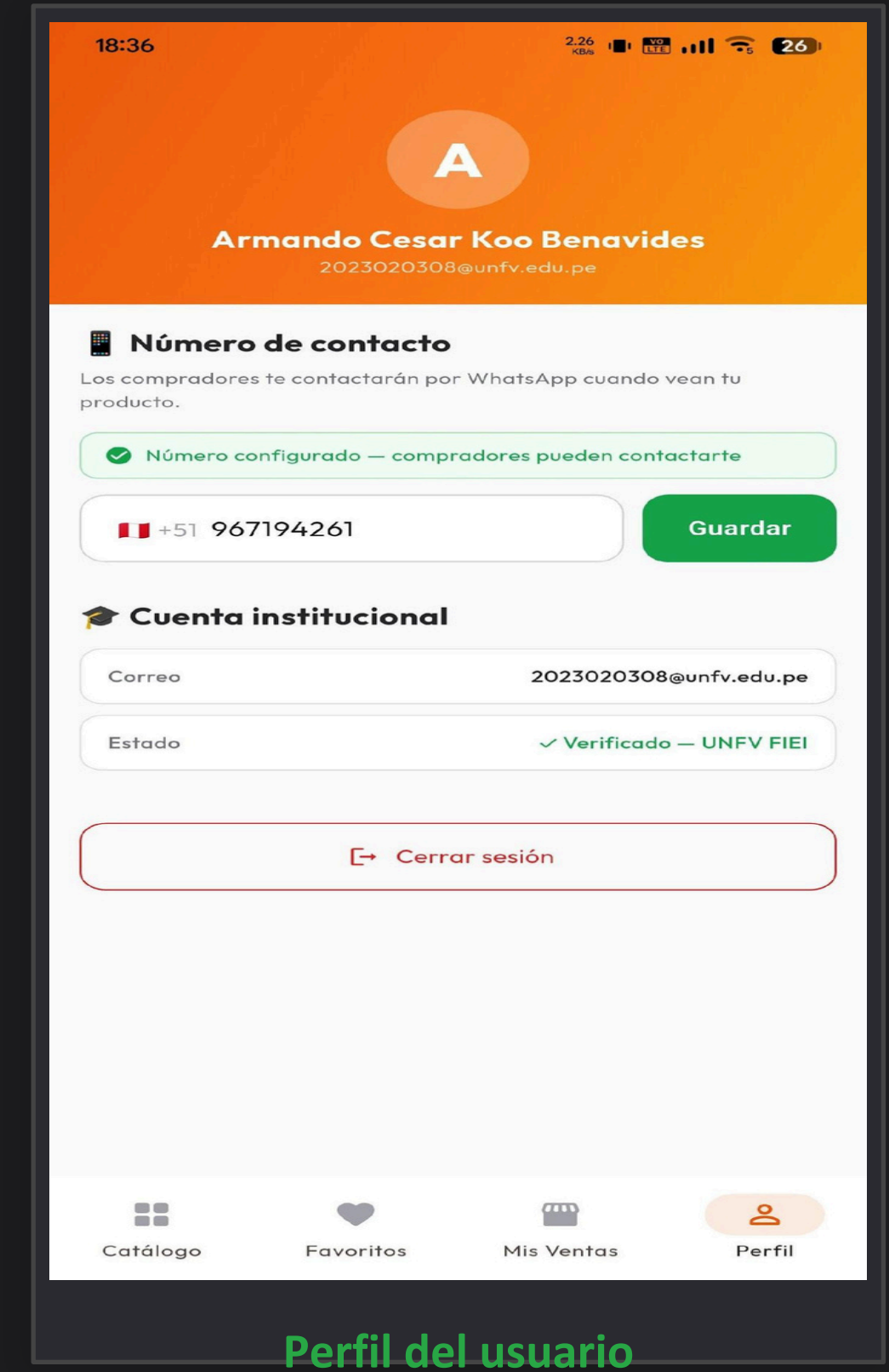
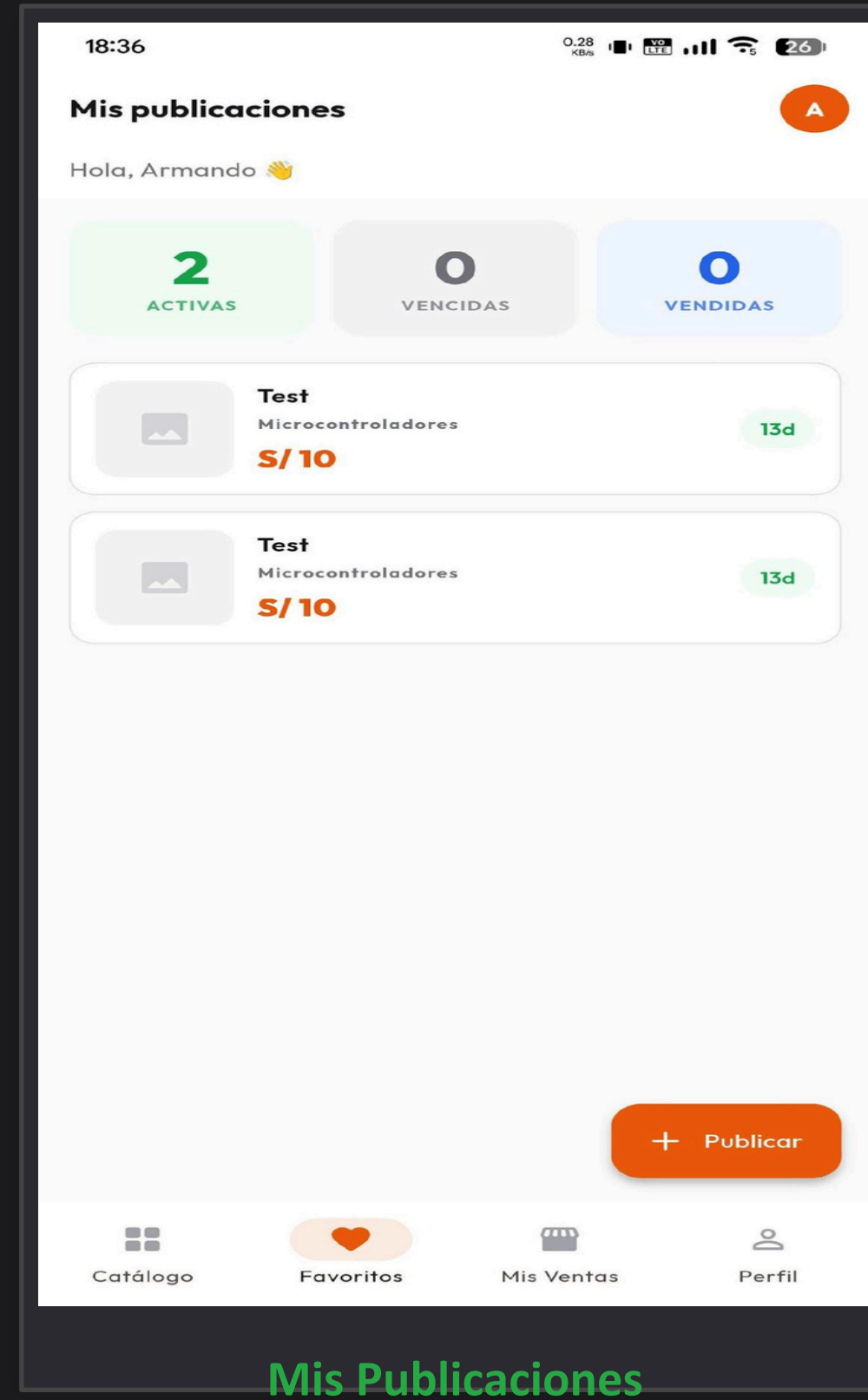
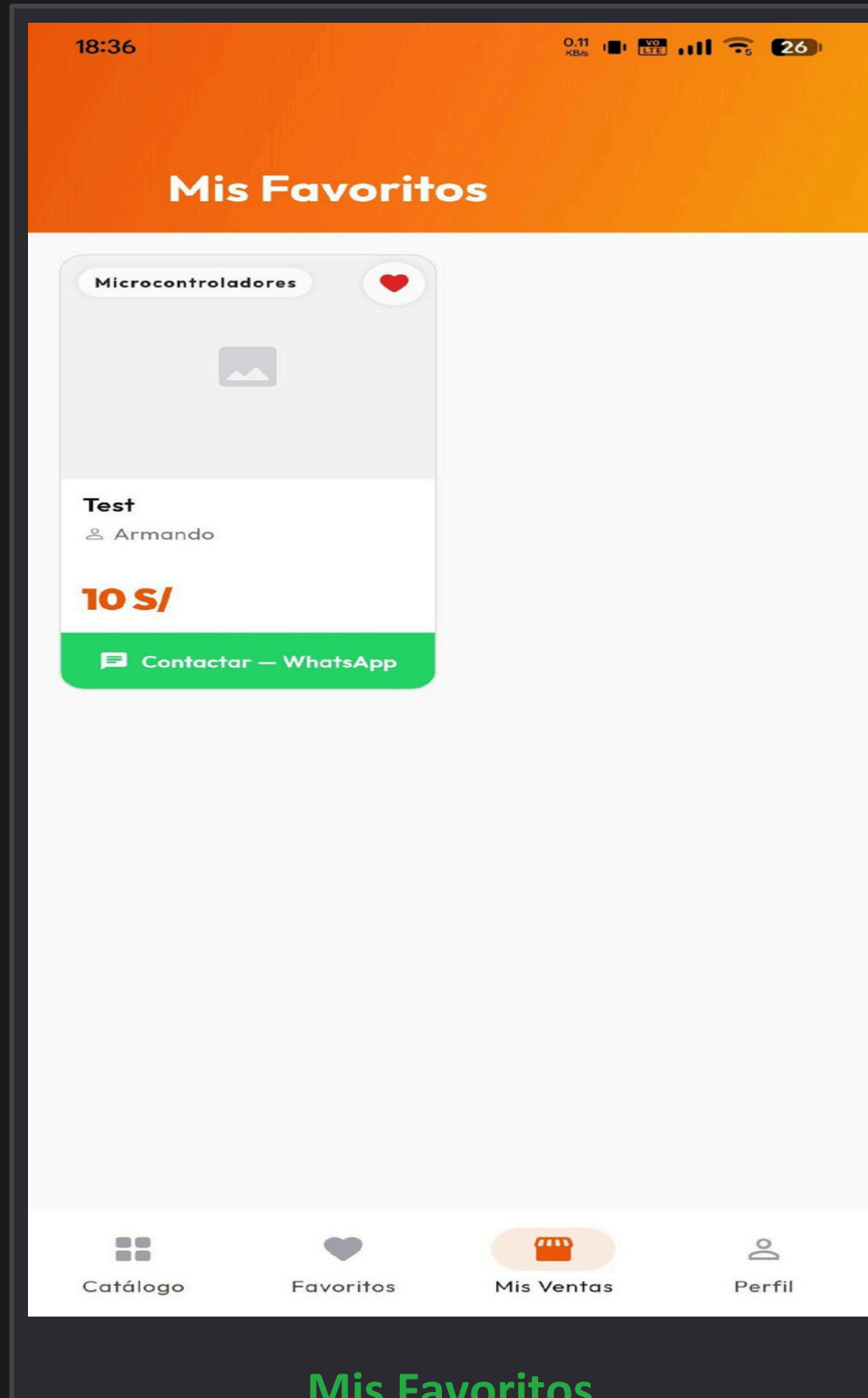
Puntos de entrega seguros

Como estudiante quiero acordar puntos de entrega seguros dentro de la universidad para facilitar el intercambio presencial.

Gestión de mis publicaciones

Como usuario quiero editar o eliminar mis publicaciones para actualizar el catálogo cuando el producto ya no esté disponible.

Sprint 2 · App en Acción



Aprendizajes y Mejoras



Aprendizajes Clave

Arquitectura en la Nube



Supabase delegó autenticación y RLS al motor BD, aligerando el frontend.



Mobile-First con Flutter

Gestión de estados complejos y peticiones asíncronas a 60fps.



Garantía Estructural

Restricciones simples (@unfv.edu.pe + puntos fijos) cambian por completo la percepción de seguridad.



Mejoras vs. Idea Inicial

01

Seguridad Nativa (RLS)

De validaciones en backend → directivas SQL imposibles de vulnerar desde el frontend.

02

Catálogo Estructurado

Categorías estrictas, escala 1-10 de condición y links a DataSheets técnicos.

03

UX Transaccional Segura

Comentarios públicos → sistema cerrado con puntos de encuentro institucionales (Patio FIEI).

Conclusión

HardSwap FIEI demuestra que una plataforma C2C institucional, respaldada por garantías estructurales (autenticación @unfv.edu.pe, RLS, puntos seguros de intercambio), transforma el comercio informal de hardware en un ecosistema confiable y eficiente dentro de la UNFV.

2

Sprints completados

8

Historias de usuario

6

Tablas en PostgreSQL

100%

Flutter + Supabase

¡Gracias! · FIEI UNFV · 2026